UNIVERSITAS MIKROSKIL | Fakultas INFORMATIKA

Feature Selection, Model Diagnosis, dan Error Analysis

IF2116 - Pembelajaran Mesin

Pertemuan 7



Sarjana Strata 1

Teknik Informatika

Materi Pertemuan 7

a **Filter, Wrapper, dan Embedded Feature Selection**

b **Model-Based Feature Selection**

c **Baseline Model dan Model Comparison**

d **Bias-Variance Trade-off**

e **Data Leakage Detection**

f **Error Analysis**

g **Reproducibility dan Dokumentasi Eksperimen**

Bekal dari Pertemuan 5 & 6

Pertemuan 5 memperkenalkan validasi silang, strategi mencegah data leakage, dan jebakan accuracy paradox pada data imbalanced. Pertemuan 6 membahas cara bekerja tanpa label (clustering & reduksi dimensi). Pertemuan 7 menyatukan semuanya menjadi satu diagnosis model: memilih fitur yang relevan, memahami dari mana error berasal, dan mendokumentasikan eksperimen secara reproducible.

a Tujuan Utama Pembelajaran

- **Memilih fitur** yang relevan menggunakan pendekatan ***filter, wrapper, dan embedded***, termasuk seleksi berbasis model (*feature importance*).
- **Membangun baseline model** yang adil sebagai pembanding, dan mengevaluasi model secara *apple-to-apple*.
- **Mendiagnosis bias-variance trade-off** melalui *learning curve* dan *validation curve* untuk menentukan kompleksitas model yang tepat.
- **Mendeteksi data leakage** dalam berbagai bentuknya (target, grup, temporal) sebelum model dipakai di dunia nyata.
- **Melakukan error analysis** secara sistematis untuk mengubah kesalahan model menjadi arah perbaikan yang konkret.
- **Mendokumentasikan eksperimen** agar hasil penelitian dapat direproduksi oleh orang lain (atau oleh diri sendiri, enam bulan kemudian).

"Model yang baik bukan yang paling akurat di leaderboard, tapi yang kita pahami betul mengapa ia benar dan mengapa ia salah."

b

KERANGKA BESAR

Dari Model yang Jalan ke Model yang Dipercaya

1

Pilih Fitur

Buang fitur yang tidak relevan / redundan (a, b)

→

2

Bandingkan dengan Baseline

Pastikan model benar-benar lebih baik dari tebakan naif (c)

→

3

Diagnosis Bias-Variance

Cari kompleksitas model yang pas, bukan under/overfit (d)

→

4

Audit Leakage & Error

Pastikan skor valid, lalu pelajari pola kesalahan (e, f)

Mengapa urutan ini penting?

Memilih fitur sebelum mendiagnosis bias-variance membuat diagnosis lebih bersih (lebih sedikit noise). Mengecek leakage sebelum error analysis mencegah kita menganalisis “kesalahan” yang sebenarnya cuma artefak kebocoran data.

Benang merah: setiap tahap di atas menjawab satu pertanyaan yang sama: “apakah skor model ini bisa dipercaya, dan mengapa?” Itulah inti dari model diagnosis.

Dataset yang dipakai: Breast Cancer Wisconsin (klasifikasi, 30 fitur) & California Housing (regresi) — konsisten dengan pertemuan sebelumnya.

Filter, Wrapper, dan Embedded Feature Selection

UNIVERSITAS MIKROSKIL

a Mengapa Feature Selection Penting?

- ***Curse of dimensionality***: makin banyak fitur, makin banyak data dibutuhkan agar model tidak overfit.
- **Generalisasi lebih baik**: fitur yang tidak relevan menambah noise dan variance pada model.
- **Interpretabilitas**: model dengan 8 fitur bermakna lebih mudah dijelaskan ke stakeholder daripada model dengan 200 fitur.
- **Efisiensi komputasi**: training & inference lebih cepat, biaya penyimpanan data lebih kecil.
- **Mengurangi risiko leakage**: fitur yang "kebetulan" bocor dari label lebih mudah terlihat saat set fitur lebih kecil dan diaudit satu per satu.

30

fitur mentah pada dataset Breast Cancer Wisconsin

569

sampel pasien (risiko overfit jika fitur terlalu banyak)

Analogi: *mencari jarum di tumpukan jerami lebih mudah kalau tumpukan jeraminya kecil. Fitur yang relevan adalah jarumnya.*

Prinsip umum: pilih fitur sesedikit mungkin lalu jaga performa model, bukan hasil sebanyak mungkin yang tersedia.

a

TAKSONOMI

Tiga Metode Feature Selection

Filter

- Menilai fitur secara independen dari model (statistik: korelasi, ANOVA F-test, chi-square)
- Sangat cepat, cocok untuk data berdimensi sangat tinggi
- Tidak memperhitungkan interaksi antar-fitur atau model yang dipakai

Wrapper

- Mencoba subset fitur dengan melatih model berulang kali (RFE, forward/backward selection)
- Memperhitungkan interaksi antar-fitur → performa seleksi lebih baik
- Mahal secara komputasi, rawan overfit ke validation set jika tidak hati-hati

Embedded

- Seleksi fitur terjadi menyatu dengan proses training (Lasso L1, feature_importances_ pada tree)
- Lebih murah dari wrapper, tetap memperhitungkan model
- Hasil seleksi spesifik untuk algoritma yang dipakai

Kapan pakai yang mana?

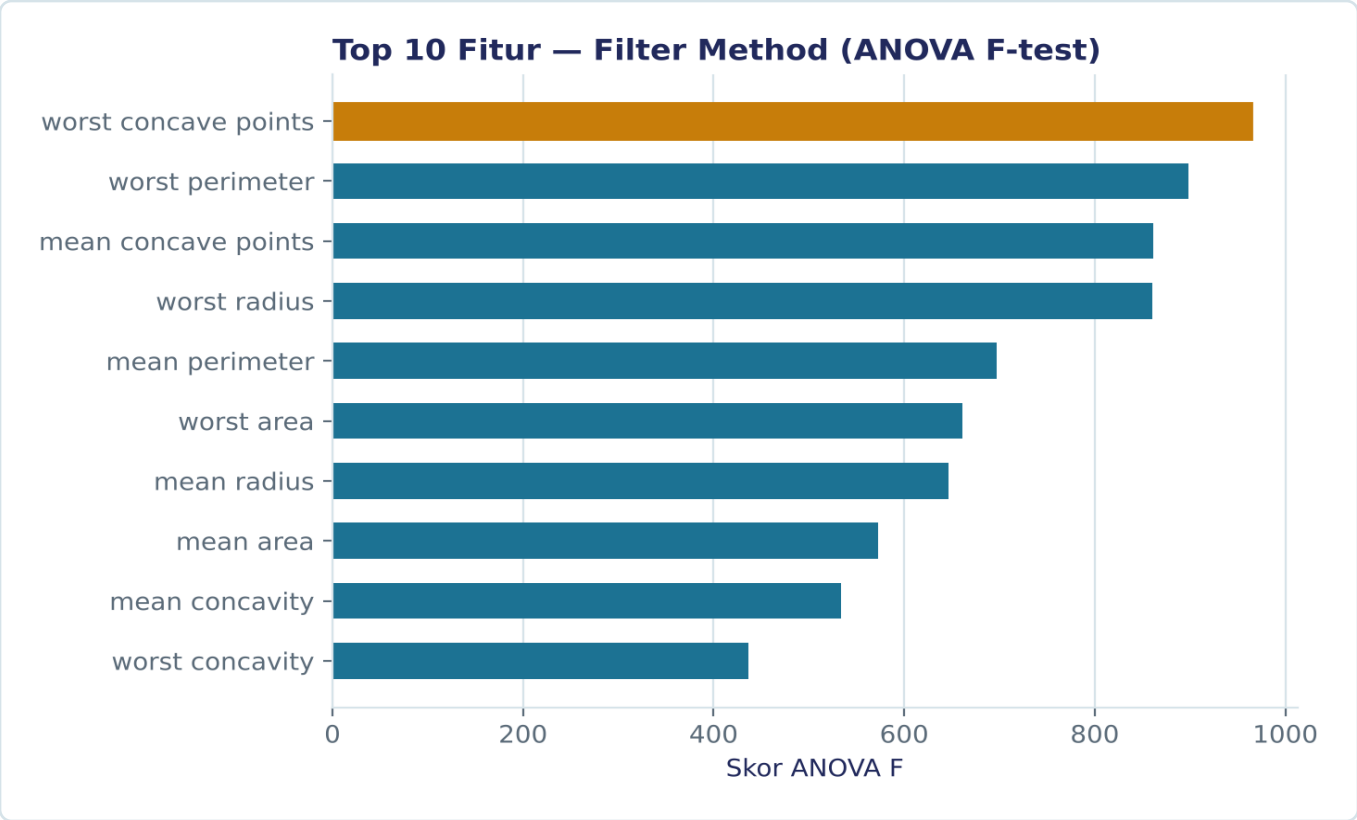
Filter untuk eliminasi cepat di tahap awal (data besar) → Wrapper saat jumlah fitur sudah lebih kecil dan butuh subset optimal → Embedded saat model akhir sudah dipilih dan ingin seleksi “gratis” selama training.

a

FILTER METHODS

Filter: Menilai Fitur Secara Statistik

- **Variance Threshold**: buang fitur yang nilainya hampir konstan (variansi rendah → info minim).
- **Correlation dengan target**: untuk fitur numerik vs target numerik (regresi).
- **ANOVA F-test** *(f_classif)*: untuk fitur numerik vs target kategorikal (klasifikasi)
- **Chi-square test**: untuk fitur kategorikal/non-negatif vs target kategorikal.
- **Mutual Information**: menangkap hubungan non-linear, lebih umum dari korelasi.



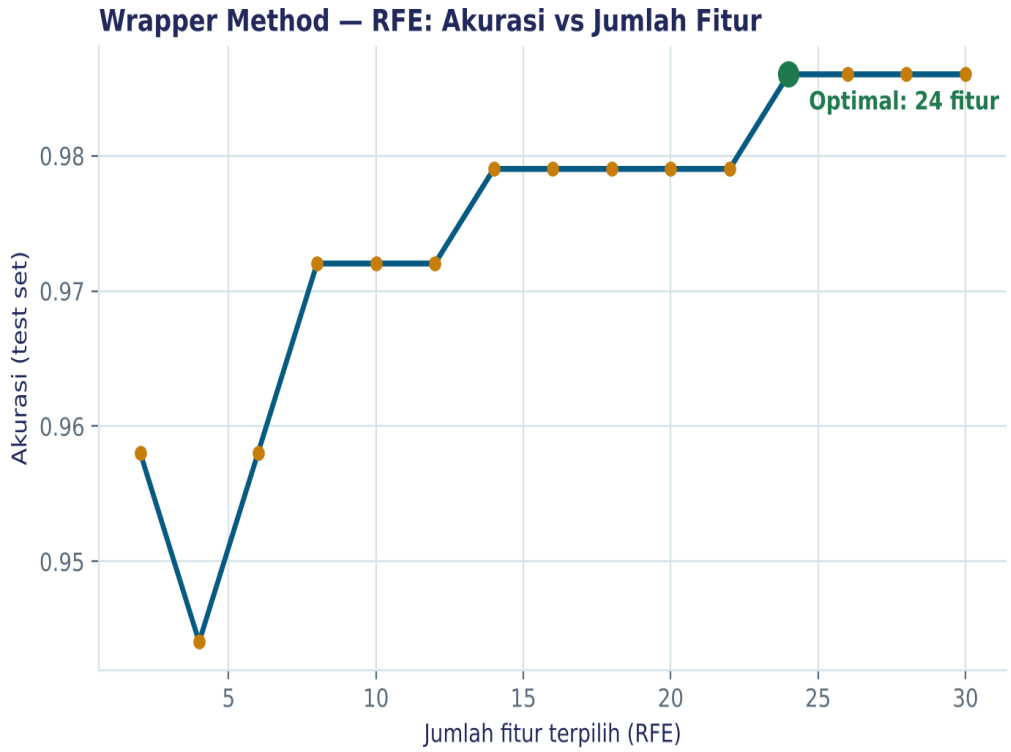
Catatan: skor filter tinggi tidak otomatis berarti fitur “bagus” untuk model non-linear, namun selalu validasi dengan performa model akhir.

Wrapper: Recursive Feature Elimination (RFE)

RFE — SCIKIT-LEARN

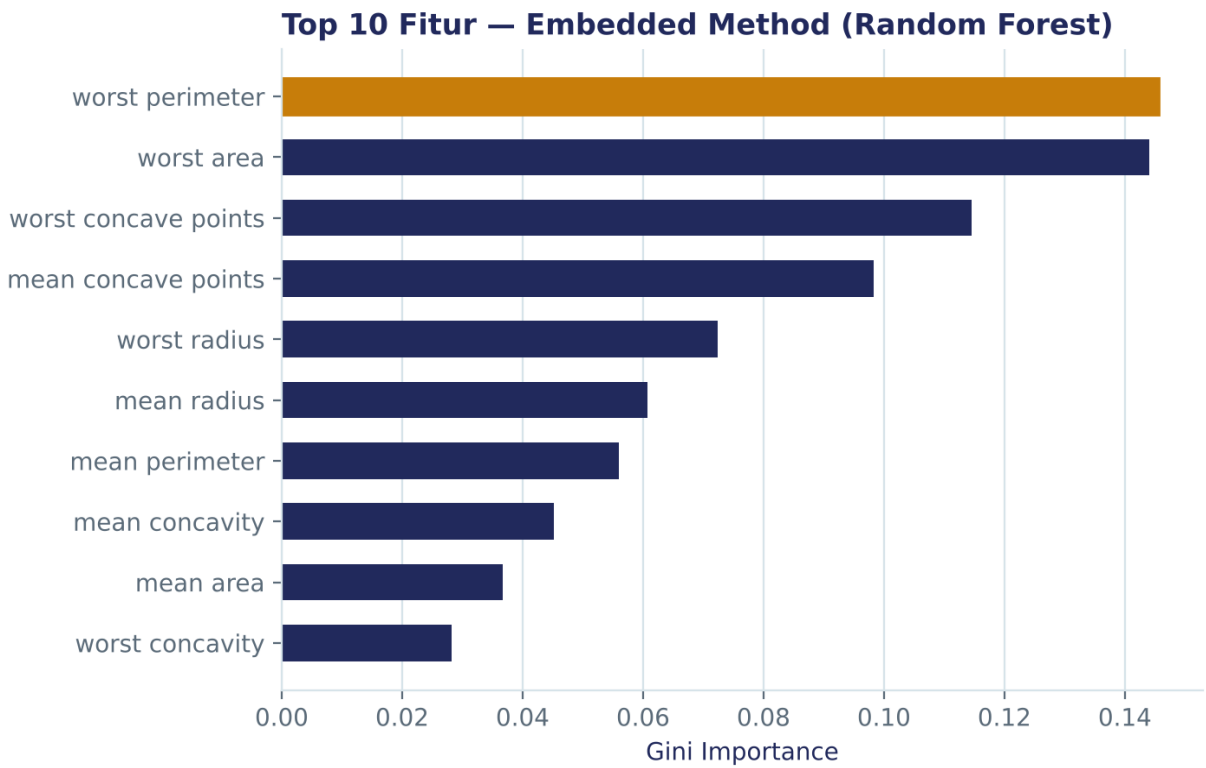
```
from sklearn.feature_selection import RFE
from sklearn.linear_model import LogisticRegression

rfe = RFE(
    estimator=LogisticRegression(max_iter=2000),
    n_features_to_select=24
)
rfe.fit(X_train_scaled, y_train)
selected = X.columns[rfe.support_]
# RFE latih ulang & buang fitur terlemah
```



- RFE melatih model, mengurutkan fitur berdasar bobot/importance, lalu membuang yang paling lemah dan diulang kembali hingga tersisa jumlah fitur target.
- Forward Selection: mulai dari 0 fitur, tambahkan satu per satu yang paling meningkatkan skor.
- Backward Elimination: mulai dari semua fitur, buang satu per satu yang paling tidak membantu.

a Embedded Methods



EMBEDDED — SELECTFROMMODEL

```
from sklearn.feature_selection import (
    SelectFromModel)
from sklearn.ensemble import (
    RandomForestClassifier)
rf = RandomForestClassifier(300)
rf.fit(X_train, y_train)
sfm = SelectFromModel(rf, threshold="median")
X_sel = sfm.transform(X_train)
```

Lasso (L1) untuk model linear

Regularisasi L1 mendorong sebagian koefisien tepat menjadi nol sehingga fitur dengan koefisien nol otomatis "tereliminasi" saat training.

Ingat: feature_importances_ bisa bias terhadap fitur dengan kardinalitas tinggi atau yang saling berkorelasi.

a Memilih Strategi Feature Selection yang Tepat

Kriteria	Filter	Wrapper	Embedded
Biaya komputasi	Sangat rendah	Tinggi	Sedang
Pertimbangkan interaksi fitur?	Tidak	Ya	Ya (sebagian)
Bergantung pada model?	Tidak	Ya	Ya
Cocok untuk	Eliminasi awal, data besar	Subset fitur optimal, data sedang	Alur kerja end-to-end
Contoh	ANOVA F-test, Chi-square	RFE, Forward/Backward	Lasso, feature_importances_

Praktik terbaik: gabungkan metode, mulai dengan filter cepat untuk membuang fitur jelas tidak berguna, lalu gunakan wrapper/embedded pada subset yang tersisa.

Model-Based Feature Selection

UNIVERSITAS MIKROSKIL

b

DARI EMBEDDED KE MODEL-BASED

Feature Importance dari Tree-Based Models

- **Gini importance** (mean decrease in impurity): seberapa sering & efektif sebuah fitur dipakai untuk memisahkan data di semua pohon Random Forest.
- **Dihitung otomatis** saat training - tidak perlu proses tambahan (atribut *.feature_importances_*).
- **Masalah: bias terhadap fitur kardinalitas tinggi** - fitur numerik dengan banyak nilai unik punya lebih banyak "kesempatan" jadi split point, sehingga importance-nya cenderung terlihat tinggi walau tidak benar-benar prediktif.
- **Masalah: fitur yang saling berkorelasi "berbagi" importance** - dua fitur yang sangat mirip bisa sama-sama terlihat kurang penting, padahal gabungannya sangat prediktif.

Analogi: *dua karyawan kembar identik mengerjakan tugas yang sama persis. Terjadi penilaian kinerja individunya bisa tampak rendah, padahal berdua mereka sangat berharga bagi tim.*

Solusi: jangan hanya mengandalkan Gini importance. Validasi dengan metode yang tidak bias terhadap struktur pohon — permutation importance.

b

PERMUTATION IMPORTANCE

Mengukur Pentingnya Fitur Tanpa Bias Struktur Model

PERMUTATION IMPORTANCE

```
from sklearn.inspection import (
    permutation_importance)

result = permutation_importance(
    rf, X_test, y_test,
    n_repeats=20, random_state=42
)

# Acak 1 kolom fitur, ukur penurunan
# skor model -> itulah importance-nya
imp_mean = result.importances_mean
```

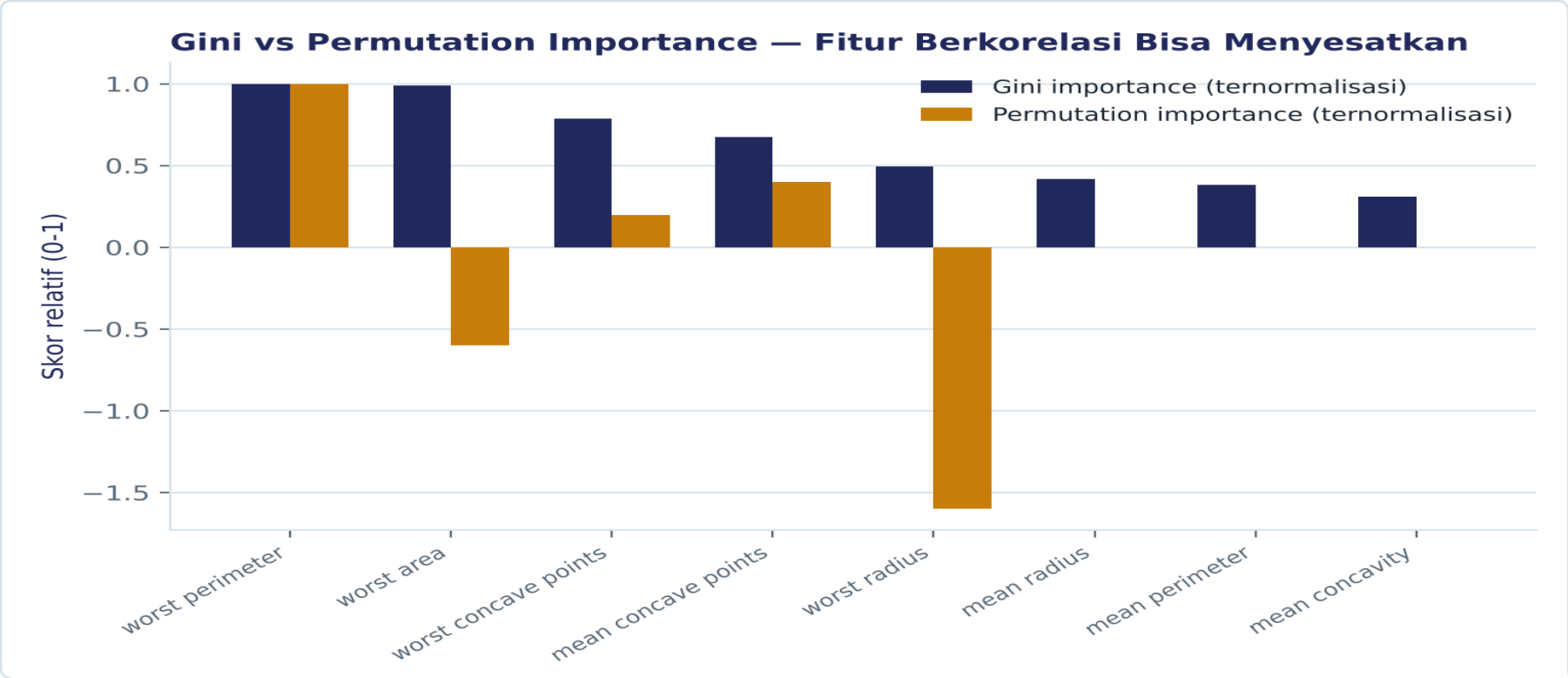
- Caranya: acak nilai satu kolom fitur (hilangkan hubungannya dengan target), lalu ukur seberapa besar skor model turun.
- Penurunan besar = fitur penting. Penurunan kecil/nol = fitur tidak banyak membantu prediksi.
- Bisa dihitung untuk model apa pun (tidak hanya tree-based) dan dievaluasi pada data test, lebih jujur merefleksikan kegunaan fitur yang sesungguhnya.

Biaya: permutation importance butuh beberapa kali prediksi ulang per fitur, sehingga lebih mahal dari Gini importance, tapi jauh lebih bisa dipercaya.

b

STUDI KASUS

Gini vs Permutation Importance pada Breast Cancer Wisconsin



Perhatikan: peringkat fitur bisa berubah antara dua metode — fitur yang tampak dominan di Gini importance belum tentu sepenting itu secara permutation importance.

b Menyusun Pipeline Seleksi Fitur Berbasis Model

PIPELINE END-TO-END

```
from sklearn.pipeline import Pipeline
from sklearn.feature_selection import (
    SelectFromModel)

pipe = Pipeline([
    ("select", SelectFromModel(
        RandomForestClassifier(300),
        threshold="median")),
    ("clf", LogisticRegression(max_iter=2000)),
])

# seleksi fitur terjadi HANYA di dalam
# setiap fold cross_val_score(pipe, cv=5)
```

Kenapa dibungkus Pipeline?

Jika seleksi fitur dilakukan di luar cross-validation, informasi dari fold validasi “bocor” ke proses pemilihan fitur, sehingga bentuk data leakage.

Pipeline
mencegah leakage otomatis

cv=5
seleksi diulang tiap fold

Spa pun yang “belajar” dari data (termasuk seleksi fitur) harus berada di dalam Pipeline yang di-fit ulang pada setiap fold, bukan dilakukan sekali di awal pada seluruh data.

Baseline Model dan Model Comparison

UNIVERSITAS MIKROSKIL

c Accuracy Paradox: Kilas Balik

Yang sudah kita pelajari di Pertemuan 5

Pada data imbalanced, model yang selalu menebak kelas mayoritas bisa mencapai akurasi 98,9% tanpa belajar apa pun. Inilah accuracy paradox. Solusinya: evaluasi dengan precision, recall, F1-score, dan balanced accuracy, bukan akurasi saja.

- Pertemuan 7 memperluas ide ini: bukan hanya untuk kasus imbalanced, tapi sebagai disiplin umum sehingga setiap model HARUS dibandingkan dengan baseline sebelum diklaim “bagus”.
- Baseline juga membantu mendeteksi bug: jika model kompleks skornya sama/lebih buruk dari baseline, kemungkinan ada masalah pada pipeline, bukan pada datanya.

Baseline bukan sekadar pembanding akurasi, ia adalah alat diagnosis pertama yang wajib dijalankan di setiap eksperimen baru.

c Jenis Baseline

DummyClassifier (klasifikasi)

- strategy="most_frequent": selalu prediksi kelas mayoritas
- strategy="stratified": prediksi acak mengikuti proporsi kelas pada data latih
- strategy="uniform": prediksi acak merata ke semua kelas
- strategy="prior": prediksi probabilitas sesuai distribusi kelas

DummyRegressor (regresi)

- strategy="mean": selalu prediksi rata-rata target data latih
- strategy="median": selalu prediksi median target
- strategy="quantile": prediksi kuantil tertentu
- Berguna untuk baseline $R^2 = 0$ pada kasus regresi

```
from sklearn.dummy import DummyClassifier
baseline = DummyClassifier(strategy="most_frequent").fit(X_train, y_train)
print("Baseline accuracy:", baseline.score(X_test, y_test))
```

c Membandingkan Model Secara Apple-to-Apple

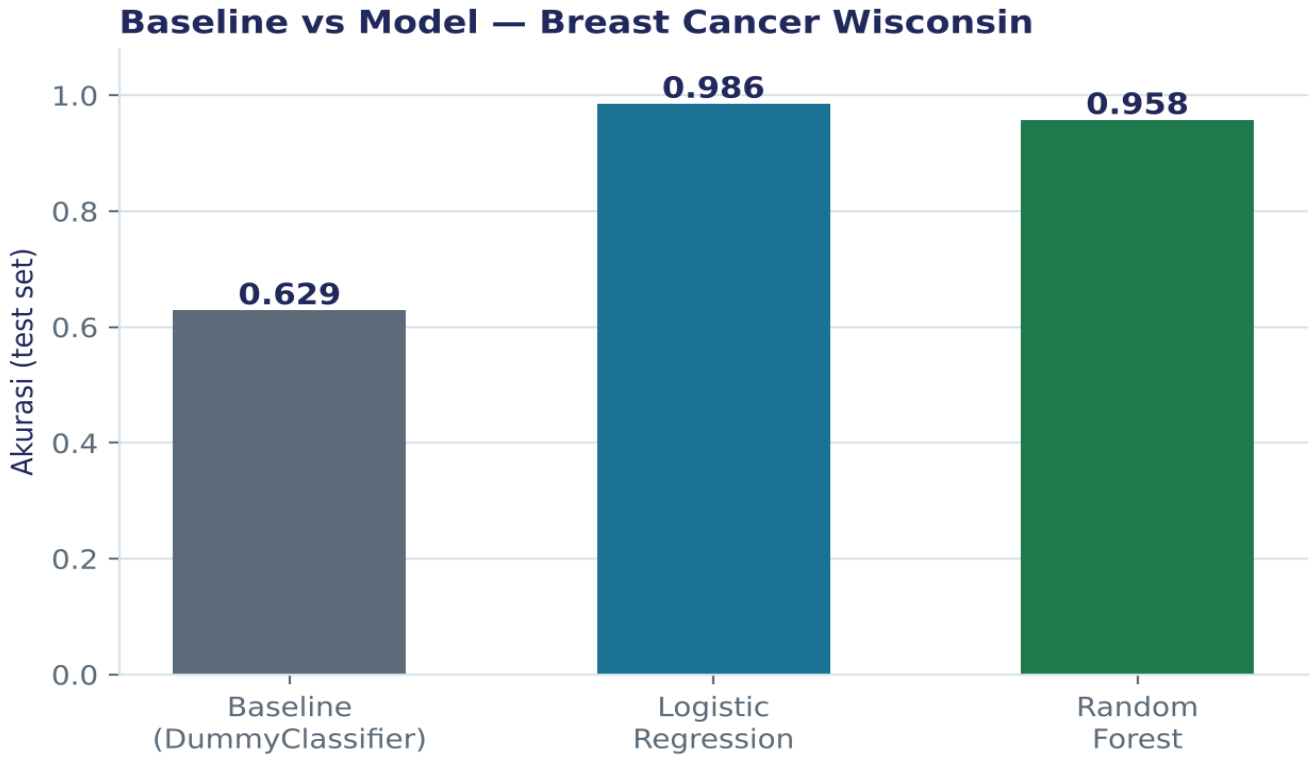
- **Gunakan data split / fold yang sama persis** untuk semua model yang dibandingkan (set random_state tetap).
- **Bandingkan distribusi skor cross-validation**, bukan hanya satu angka — model dengan rata-rata lebih tinggi tapi simpangan baku sangat besar belum tentu “lebih baik”.
- **Pertimbangkan signifikansi praktis**: selisih 0,1% akurasi mungkin tidak berarti apa-apa secara praktis meski “terlihat” lebih tinggi.
- **Bandingkan dari berbagai sudut**: metrik utama, waktu training, interpretabilitas, dan kebutuhan komputasi saat deployment.

BANDINGKAN DENGAN CROSS-VALIDATION YANG SAMA

```
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
for name, model in [("baseline", baseline), ("logreg", logreg), ("rf", rf)]:
    scores = cross_val_score(model, X_train, y_train, cv=cv, scoring="accuracy")
    print(f"{name}: {scores.mean():.3f} +/- {scores.std():.3f}")
```

STUDI KASUS

c Baseline vs Model pada Breast Cancer Wisconsin



+35.6 poin

peningkatan akurasi: baseline 62,9% → Logistic Regression 98,6%

Diagnosis

Selisih besar dari baseline menunjukkan model benar-benar belajar pola bermakna dari fitur — bukan hanya menghafal distribusi kelas. Ini memberi keyakinan bahwa pipeline berjalan sesuai harapan.

Aturan : laporkan skor baseline berdampingan dengan skor model di setiap laporan eksperimen, bukan opsional.

Bias-Variance Trade-off

UNIVERSITAS MIKROSKIL

d Bias vs Variance: Dua Sumber Error yang Berbeda

KONSEP DASAR

Bias tinggi

- Model terlalu sederhana untuk menangkap pola data
- Prediksi konsisten tapi jauh dari target (sistematis salah)
- Gejala: underfitting menhasilkan skor training & validasi sama-sama rendah

Variance tinggi

- Model terlalu sensitif terhadap data training spesifik
- Prediksi berubah drastis jika data training sedikit berbeda
- Gejala: overfitting menjadikan skor training tinggi, skor validasi jauh lebih rendah

Analogi: *latihan panahan. Bias tinggi = semua anak panah mengumpul jauh dari titik tengah (konsisten tapi salah arah). Variance tinggi = anak panah tersebar ke mana-mana di sekitar titik tengah (rata-rata benar, tapi tidak konsisten).*

Target : bias rendah DAN variance rendah secara bersamaan seperti anak panah mengumpul rapat tepat di titik tengah.

d Total Error = $Bias^2$ + Variance + Irreducible Error

$$Error(x) = Bias^2 + Variance + Irreducible\ Error$$

- **$Bias^2$**: seberapa jauh rata-rata prediksi model dari nilai sebenarnya, ini dikontrol oleh kompleksitas model (semakin kompleks, semakin rendah).
- **Variance**: seberapa besar prediksi berubah antar dataset training berbeda, ini dapat dikontrol oleh kompleksitas model (semakin kompleks, semakin tinggi).
- **Irreducible error**: noise bawaan data yang tidak bisa dihilangkan model apa pun, sebaik apa pun pemodelannya.

Implikasi praktis: menambah kompleksitas model menurunkan bias tapi menaikkan variance, inilah mengapa disebut trade-off, bukan optimisasi satu arah.

d Underfitting, Overfitting, dan Fit yang Wajar

Underfitting

Model (mis. regresi linear) terlalu sederhana untuk pola data yang melengkung. Error training DAN validasi sama-sama tinggi.

Fit yang Wajar

Kompleksitas model sesuai dengan kompleksitas pola data. Error training rendah, error validasi juga rendah dan mendekati error training.

Overfitting

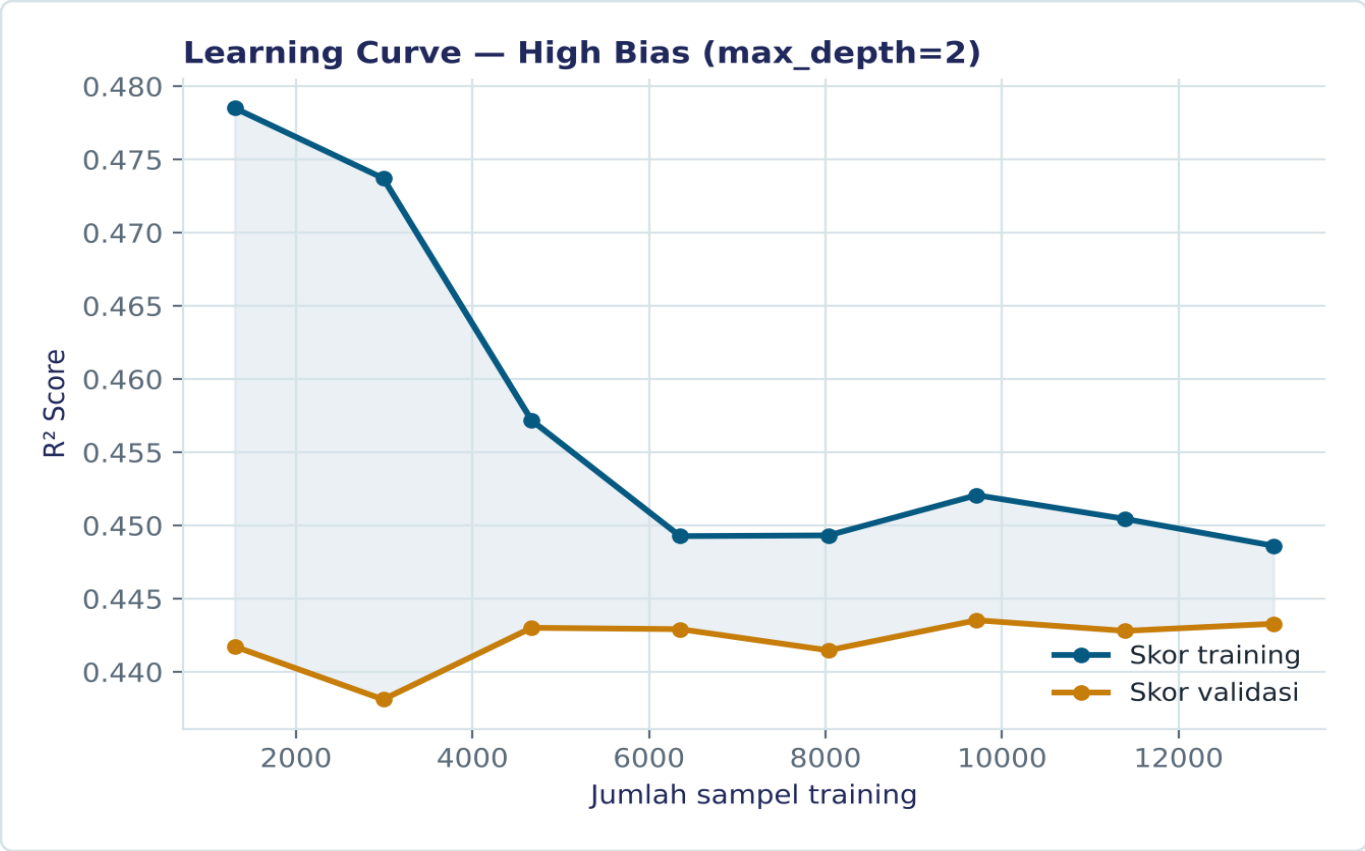
Model (mis. pohon sangat dalam) menghafal noise pada data training. Error training sangat rendah, error validasi tinggi.

Sudah dibahas ringkas di Pertemuan 5

sebagai pengantar bias-variance pada ensemble. Sekarang kita perdalam dengan alat diagnosis kuantitatif: learning curve dan validation curve.

Kunci diagnosis: lihat GAP antara skor training dan skor validasi, bukan hanya nilai mutlaknya.

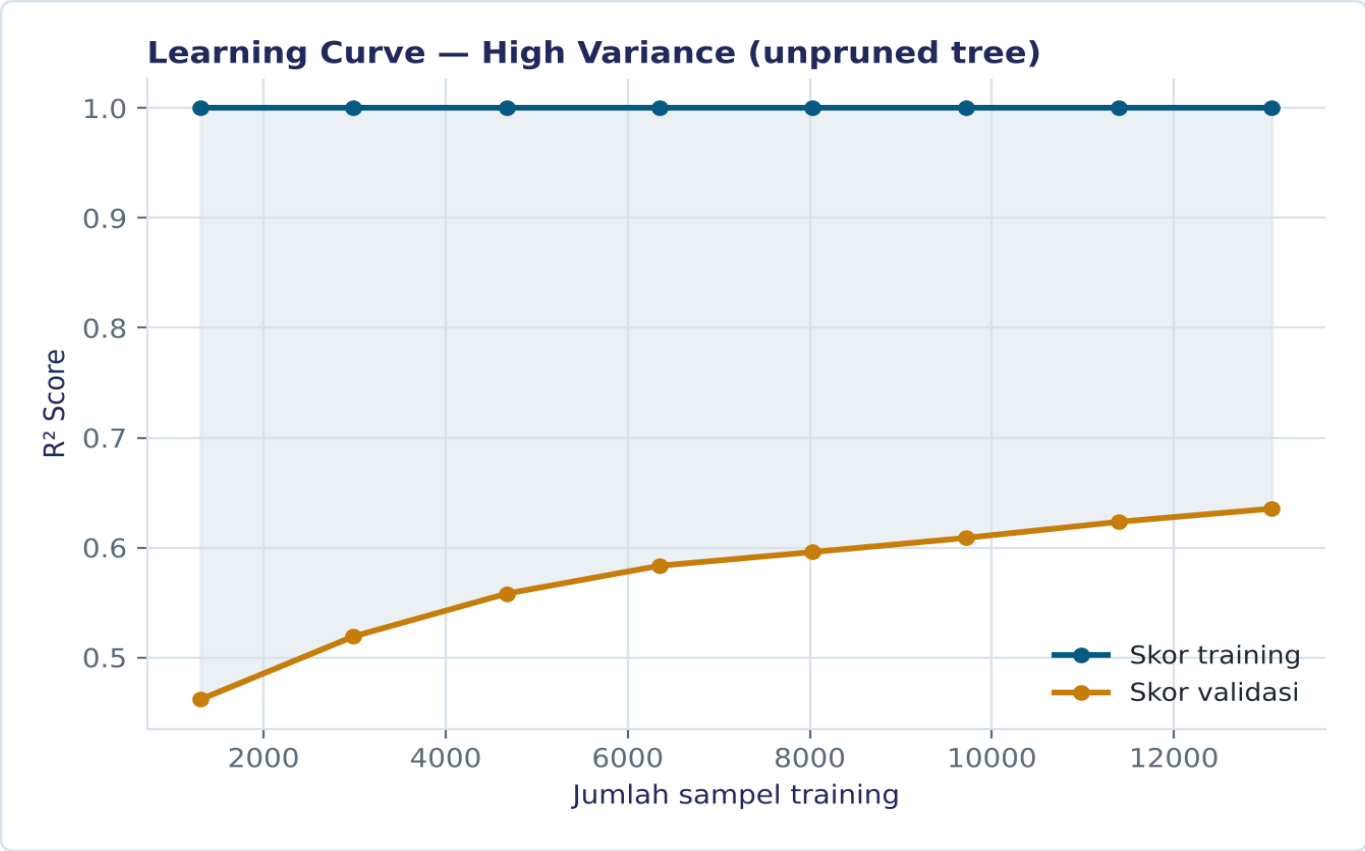
d Mendiagnosis Bias Tinggi dengan Learning Curve



- Ciri khas: kurva training & validasi konvergen ke skor yang SAMA-SAMA RENDAH.
- Menambah lebih banyak data TIDAK akan banyak membantu, masalahnya ada pada kompleksitas model, bukan jumlah data.
- Contoh (Decision Tree, max_depth=2): skor training akhir 0.449, skor validasi 0.443, keduanya rendah dan berdekatan.

Solusi: naikkan kompleksitas model, tambah fitur relevan, atau kurangi regularisasi.

d Mendiagnosis Variance Tinggi dengan Learning Curve

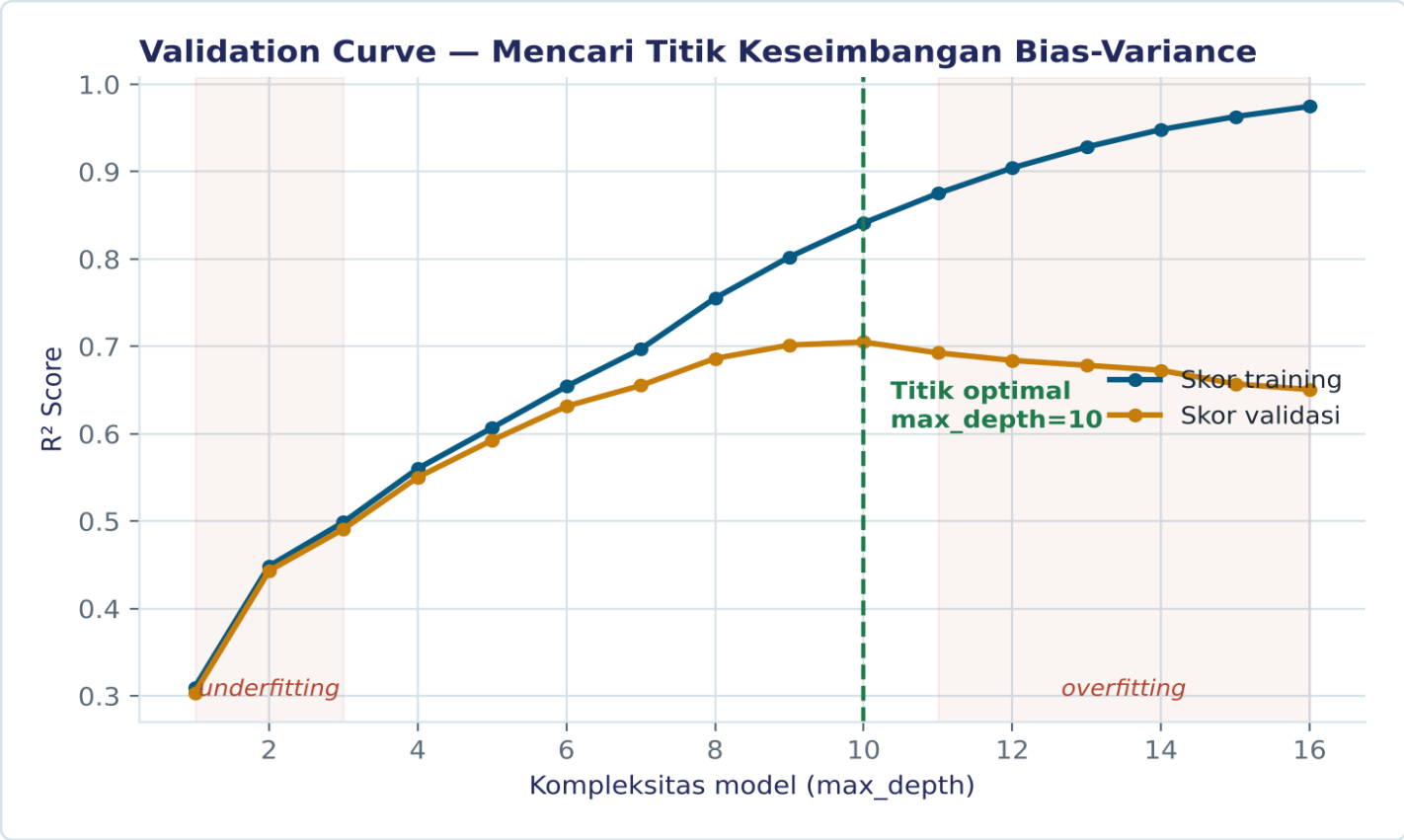


- Ciri khas: skor training SANGAT TINGGI (nyaris sempurna), tapi skor validasi jauh lebih rendah dan GAP besar.
- Menambah lebih banyak data training biasanya MEMBANTU, tetapi gap cenderung menyempit seiring data bertambah.
- Contoh (unpruned Decision Tree): skor training 1.000, skor validasi 0.635, hasil gap sangat lebar, tanda overfitting.

Solusi: tambah data, sederhanakan model, atau tambahkan regularisasi.

d

Validation Curve: Mencari Kompleksitas yang Pas



- Validation curve mengukur skor training & validasi sambil menyapu satu hyperparameter kompleksitas (mis. max_depth, C, jumlah estimator).
- Pada California Housing, titik optimal berada di sekitar max_depth=10 di kiri (dangkal) underfitting, di kanan (dalam) overfitting.
- Berbeda dari learning curve: validation curve menyapu kompleksitas model pada UKURAN DATA TETAP, bukan menyapu jumlah data.

learning curve menjawab "*butuh data lebih banyak atau tidak?*", validation curve menjawab "*seberapa kompleks model seharusnya?*"

d Mengatasi Bias Tinggi vs Variance Tinggi

Jika bias tinggi (underfitting)

- Gunakan model lebih kompleks (naikkan max_depth, tambah layer, dsb.)
- Tambah fitur baru / fitur interaksi yang relevan
- Kurangi kekuatan regularisasi (turunkan alpha/lambda)
- Latih lebih lama (lebih banyak epoch/iterasi) jika model belum konvergen

Jika variance tinggi (overfitting)

- Tambah data training, atau augmentasi data
- Sederhanakan model (turunkan max_depth, kurangi parameter)
- Tambah regularisasi (naikkan alpha/lambda, dropout, early stopping)
- Gunakan ensemble (bagging) untuk merata-ratakan variance — lihat kembali Pertemuan 5

Note: jangan menambah regularisasi pada model yang justru underfit, atau menambah kompleksitas pada model yang sudah overfit. Learning/validation curve adalah alat diagnosisnya.

Data Leakage Detection

UNIVERSITAS MIKROSKIL

e Leakage pada Data Murni Acak: Kilas Balik

Yang sudah kita pelajari di Pertemuan 5

Pada 100 sampel berlabel acak dengan 5.000 fitur noise, seleksi fitur yang dilakukan DI LUAR fold cross-validation menghasilkan akurasi palsu 1 (seharusnya ~0,50 karena label acak). Begitu seleksi fitur dipindah ke DALAM Pipeline per-fold, akurasi turun ke level yang benar (~0,51). Aturan emas: apa pun yang memanggil .fit() harus berada di dalam Pipeline yang di-fit ulang tiap fold.

- Pertemuan 7 memperluas cakupan leakage di luar seleksi fitur: target leakage, group leakage, dan temporal leakage.
- Kita juga akan membahas TEKNIK DETEKSI sistematis, bukan hanya definisi — agar leakage bisa ditemukan sebelum model dipakai di produksi.

Masih berlaku: kecurigaan pertama saat skor validasi “terlalu bagus” (mendekati sempurna) bukan euforia, melainkan dugaan leakage yang harus diaudit.

e Empat Jenis Data Leakage yang Sering Terjadi

1 Target Leakage

Fitur yang hanya tersedia SETELAH target diketahui (mis. “tanggal pembayaran tagihan” untuk memprediksi gagal bayar) ikut dipakai sebagai input.

2 Train-Test Contamination

Preprocessing (scaling, imputasi, seleksi fitur) dihitung dari SELURUH data sebelum split, sehingga info dari test set bocor ke proses training.

3 Group Leakage

Sampel dari entitas yang sama (pasien, pengguna) muncul baik di train maupun test, sehingga model “mengenali” entitasnya, bukan belajar pola umum.

4 Temporal Leakage

Pada data time-series, data “masa depan” relatif terhadap titik prediksi ikut terpakai untuk melatih model yang akan dipakai memprediksi masa lalu/sekarang.

Benang merah: semua jenis leakage pada akhirnya adalah bentuk “melihat masa depan” yang seharusnya tidak boleh dilihat saat training.

e Cara Sistematis Mendeteksi Data Leakage

- **Audit korelasi ekstrem**: fitur dengan korelasi > 0,95 terhadap target patut dicurigai — tanyakan apakah fitur itu tersedia SEBELUM prediksi dibutuhkan.
- **Audit waktu ketersediaan fitur (feature timing audit)**: untuk setiap fitur, catat kapan nilainya benar-benar diketahui relatif terhadap waktu prediksi.
- **Bandingkan skor train vs test yang "terlalu dekat & terlalu tinggi"**: performa sempurna di awal eksperimen adalah sinyal bahaya, bukan prestasi.
- **Audit duplikasi / near-duplicate antar split**: periksa apakah ID unik (pasien, pengguna, transaksi) muncul di train maupun test.

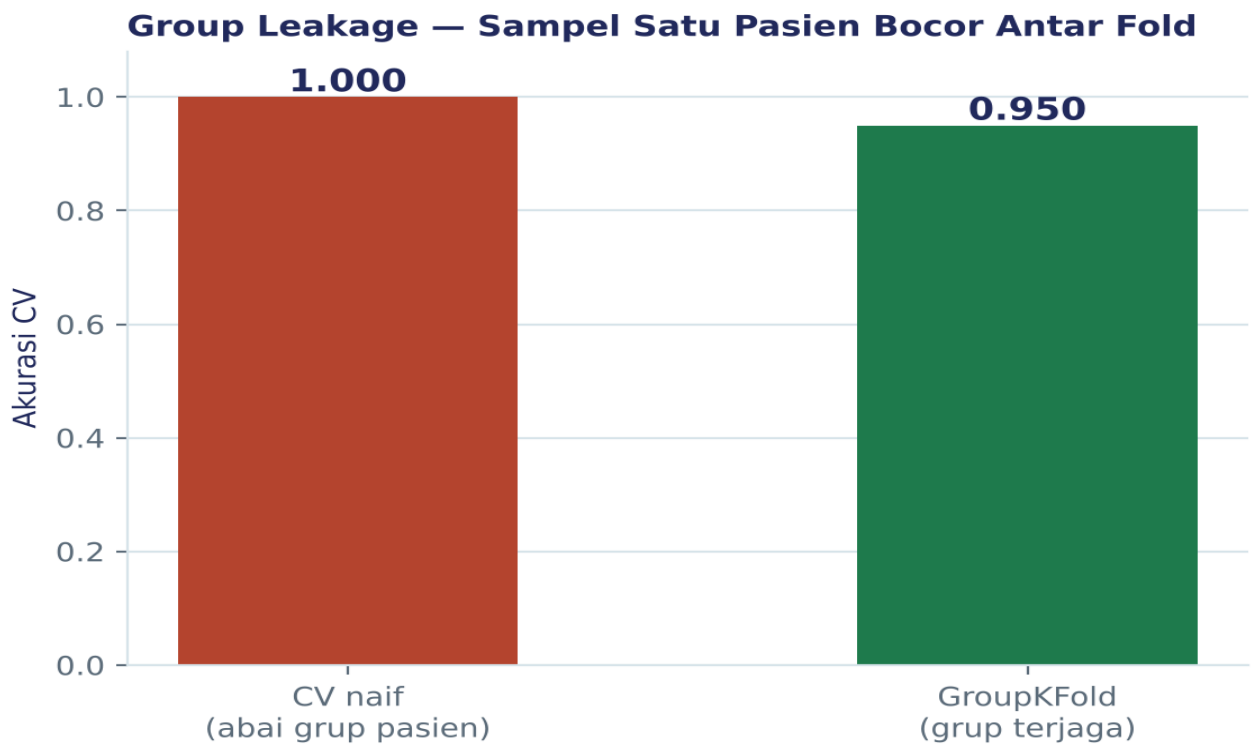
CEK DUPLIKASI GRUP ANTAR SPLIT

```
overlap = set(train_df["patient_id"]) & set(test_df["patient_id"])
assert len(overlap) == 0, f"Group leakage: {len(overlap)} pasien muncul di train & test"
```

STUDI KASUS NYATA

STUDI KASUS BARU

Group Leakage: Saat Satu Pasien “Bocor” ke Dua Sisi



Setup eksperimen

Simulasi 40 "pasien", masing-masing menyumbang 5 sampel yang hampir identik (variasi kecil). Dibandingkan: K-Fold acak biasa vs GroupKFold yang menjaga seluruh sampel satu pasien tetap berada di sisi yang sama (train ATAU test, tidak dua-duanya).

100%
akurasi CV naif (grup bocor)

95%
akurasi GroupKFold (jujur)

Deteksi: jika entitas (pasien/pengguna) punya lebih dari satu baris data, gunakan GroupKFold, bukan KFold biasa.

```
from sklearn.model_selection import GroupKFold
cross_val_score(model, X, y, cv=GroupKFold(5), groups=patient_id)
```

e Checklist Pencegahan Data Leakage

- Lakukan train-test split (atau definisikan skema CV) SEBELUM melakukan eksplorasi atau preprocessing apa pun.
- Bungkus seluruh preprocessing (scaling, imputasi, seleksi fitur, encoding) dalam satu Pipeline scikit-learn.
- Gunakan GroupKFold jika data punya struktur entitas berulang (pasien, pengguna, sesi).
- Gunakan TimeSeriesSplit (bukan KFold acak) jika data memiliki struktur waktu.
- Tinjau setiap fitur satu per satu: “apakah nilai ini sungguh-sungguh tersedia pada saat prediksi dibutuhkan?”
- Curigai skor yang nyaris sempurna dengan validasi ulang sebelum menjadikan hasil.

Aturan emas Pertemuan 5 & 7: apa pun yang memanggil .fit() atau “belajar” dari data harus berada di dalam Pipeline yang di-fit ulang per-fold, tanpa kecuali.

Error Analysis

UNIVERSITAS MIKROSKIL

f Apa Itu Error Analysis dan Mengapa Penting?

- **Error analysis** adalah proses sistematis memeriksa SAMPEL mana saja yang salah diprediksi model, dan MENCARI POLA di baliknya.
- **Berbeda dari sekadar melihat metrik agregat** (akurasi, F1) error analysis menjawab pertanyaan "jenis kesalahan apa yang paling sering terjadi, dan pada kondisi apa?"
- **Mengubah metrik menjadi arah perbaikan konkret**: bukan "akurasi 92%, lumayan", tapi "70% kesalahan terjadi pada subgrup X — mari tambah data/fitur untuk subgrup itu".
- **Praktik umum di industri (Andrew Ng dkk.)**: ambil ~100 sampel yang salah diprediksi, kategorikan manual penyebabnya, prioritaskan perbaikan pada kategori terbesar.

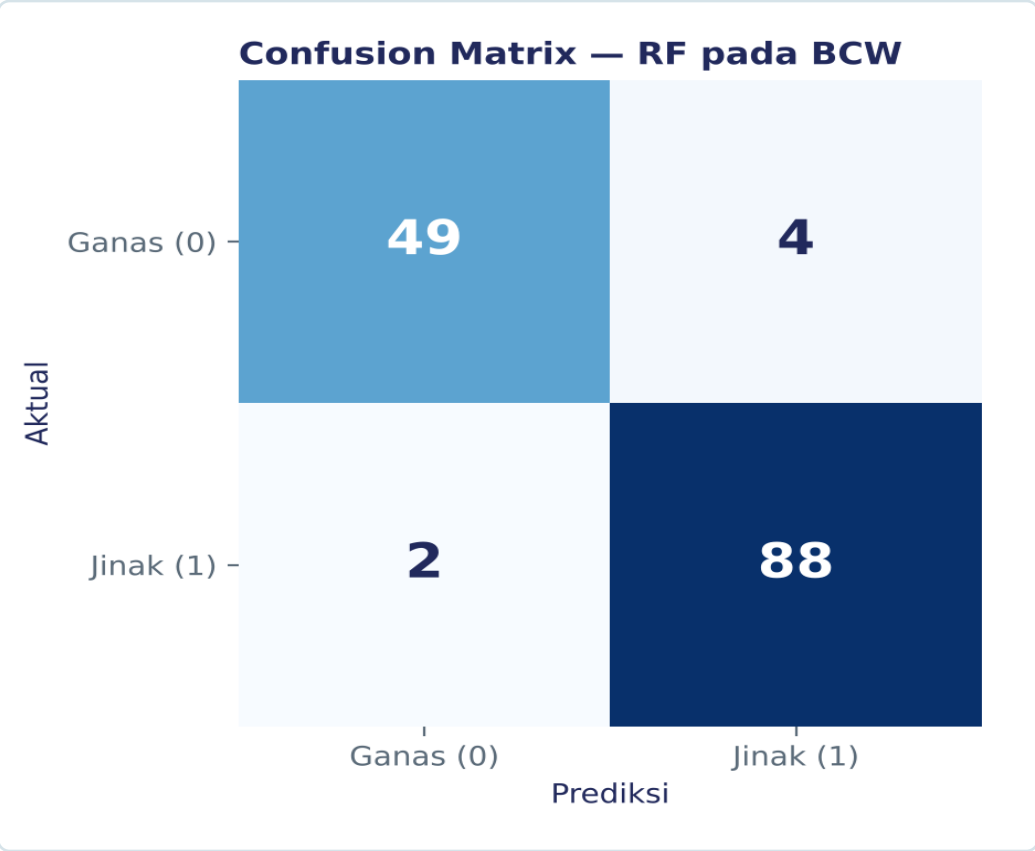
Analogi: *dokter yang tidak hanya mencatat "pasien demam", tapi mencari tahu APA penyebab demamnya (infeksi, alergi, atau lainnya) sebelum meresepkan obat.*

Prinsip: jangan memperbaiki model secara acak Tapi perbaiki berdasarkan pola kesalahan yang terbukti ada.

RECAP PERTEMUAN 5

DARI METRIK KE ANALISIS

Confusion Matrix sebagai Titik Awal Error Analysis



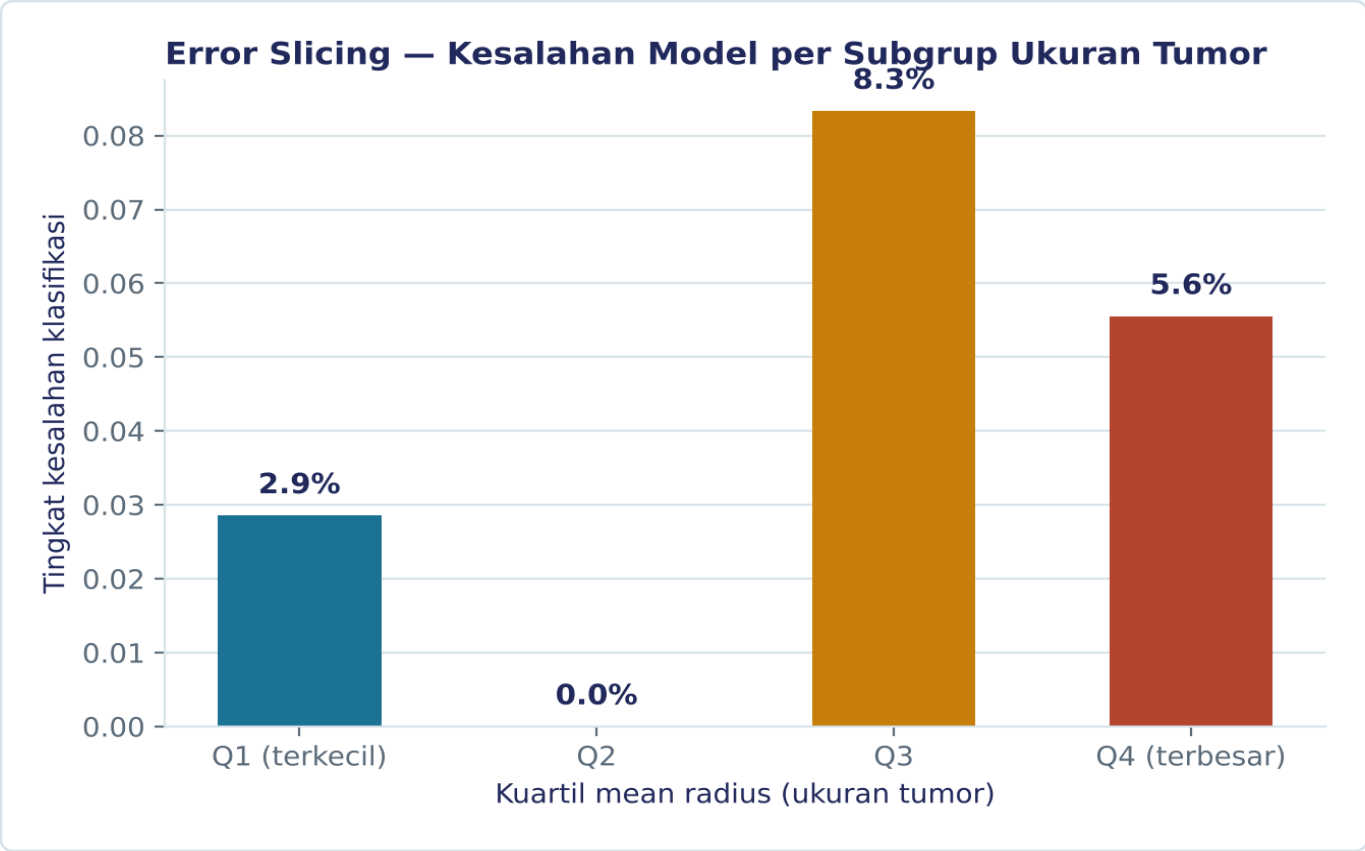
- False Negative (2 kasus): tumor ganas diprediksi jinak — PALING berbahaya secara klinis, prioritas analisis tertinggi.
- False Positive (4 kasus): tumor jinak diprediksi ganas — menyebabkan pemeriksaan lanjutan yang tidak perlu, tapi tidak fatal.
- Pertemuan 5 memperkenalkan precision/recall dari matriks ini. Pertemuan 7: kita bedah SAMPEL individual di balik sel FN & FP ini.

Langkah selanjutnya: jangan berhenti di angka 2 dan 4 — buka baris data yang menyumbang error tersebut, baca fitur-fiturnya.

STUDI KASUS NYATA

ERROR SLICING

Memecah Error per Subgrup: Apakah Model Adil di Semua Kondisi?

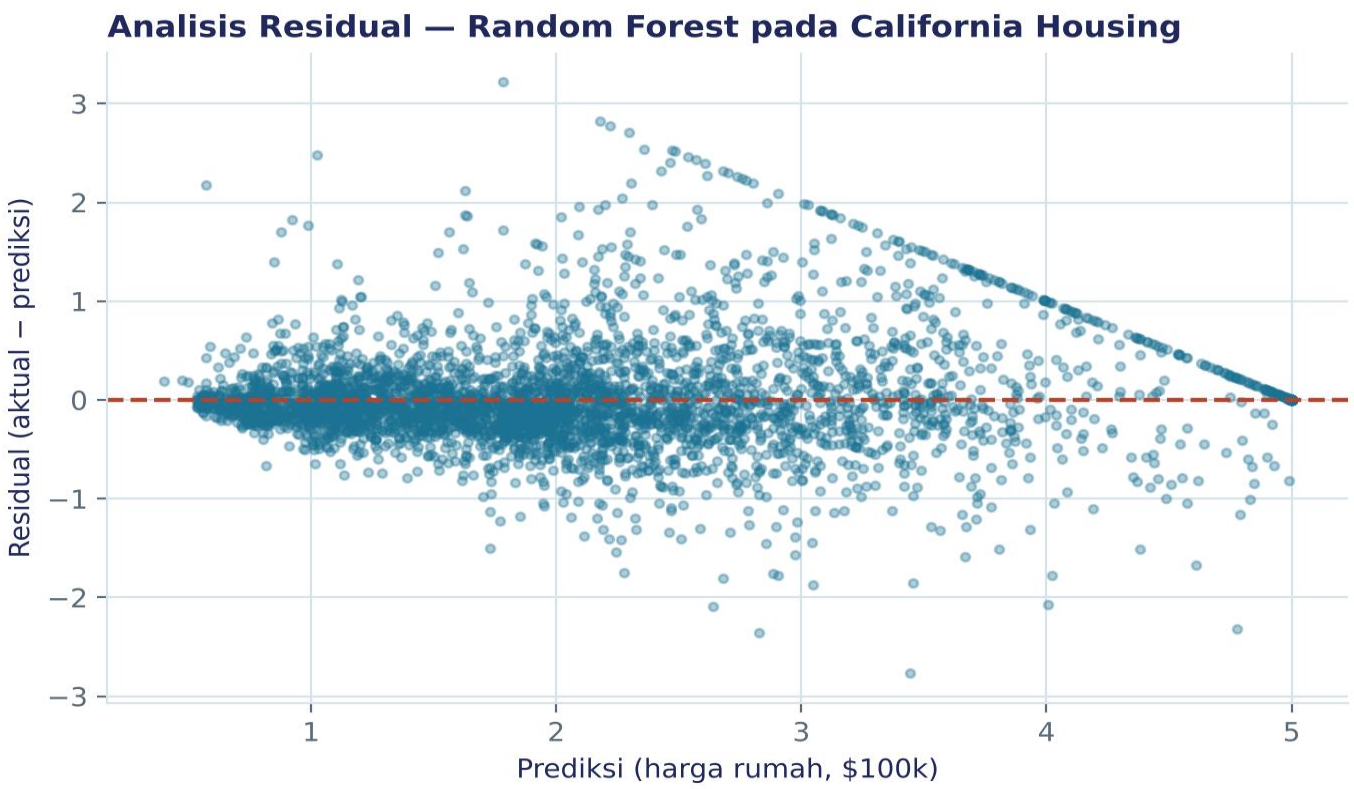


- Error slicing membagi data test ke beberapa subgrup (mis. kuartil ukuran tumor) dan menghitung tingkat kesalahan per subgrup secara terpisah.
- Pada contoh ini, tumor berukuran PALING BESAR (Q4) punya tingkat kesalahan lebih tinggi — kemungkinan karena sampel ekstrem lebih jarang di data training.
- Pola ini TIDAK terlihat dari akurasi keseluruhan — baru muncul setelah data dipecah per subgrup.

Praktik penting: selalu coba slicing berdasar atribut yang relevan secara domain (usia, wilayah, ukuran, waktu) — bukan hanya metrik agregat.

f ERROR ANALYSIS REGRESI

Analisis Residual pada Model Regresi



- **Residual = aktual − prediksi**. Plot residual vs prediksi membantu melihat pola error yang tidak tertangkap metrik tunggal seperti R^2 atau RMSE.
- **Pola acak di sekitar garis nol** = bagus, model tidak punya bias sistematis.
- **Pola berbentuk corong/kurva** = model kehilangan pola tertentu pada rentang nilai tertentu (sering pada harga rumah ekstrem tinggi).
- Pada contoh ini: R^2 = 0.8063, simpangan baku residual = $51k.

Ingat: residual besar pada nilai target ekstrem sering menandakan data training kurang representatif di area tersebut, bukan sekadar “model kurang canggih”.

f SIKLUS PERBAIKAN

Dari Error Analysis ke Perbaikan Model: Siklus Iteratif

1

Kumpulkan sampel salah

Ambil ~50-100 sampel yang diprediksi salah dari test/validation set

→

2

Kategorikan manual

Beri label penyebab tiap kesalahan (data kotor, fitur kurang, kelas ambigu, dll.)

→

3

Hitung proporsi tiap kategori

Urutkan dari penyebab terbesar — fokuskan usaha di situ

→

4

Perbaiki & ulangi

Tambah fitur/data untuk kategori terbesar, latih ulang, analisis error lagi

Perbedaan dengan tuning hyperparameter: error analysis mengarahkan KITA ke akar masalah (data/fitur), sementara tuning hanya mengoptimalkan model dengan data & fitur yang sama.

RANGKUMAN TEMUAN

f Contoh Laporan Error Analysis

Kategori Penyebab	Proporsi	Contoh & Tindak Lanjut
Sampel pada rentang nilai ekstrem (tumor besar / rumah mahal)	~38%	Kumpulkan lebih banyak data pada rentang ini, atau tambah fitur yang lebih informatif
Fitur yang hilang/tidak lengkap	~24%	Perbaiki pipeline imputasi; audit sumber data
Kelas/kasus ambigu secara inheren	~19%	Tinjau ulang definisi label bersama pakar domain
Kesalahan label pada data training (noisy label)	~12%	Audit ulang sebagian data training, bersihkan label
Lainnya / tidak teridentifikasi polanya	~7%	Butuh investigasi lanjutan dengan sampel lebih banyak

Format laporan: setiap laporan eksperimen sebaiknya menyertakan tabel seperti ini, bukan hanya satu angka metrik akhir.

Reproducibility dan Dokumentasi Eksperimen

UNIVERSITAS MIKROSKIL

g Mengapa Reproducibility Penting dalam Machine Learning?

- **Validasi ilmiah**: klaim “model saya mencapai akurasi 95%” hanya bermakna jika orang lain bisa memverifikasinya dengan langkah yang sama.
- **Debugging & kolaborasi tim**: rekan kerja (atau diri sendiri di masa depan) perlu bisa menjalankan ulang eksperimen yang sama persis untuk melanjutkan atau memperbaikinya.
- **Sumber ketidakkonsistenan yang umum**: random seed yang tidak di-fix, versi library berbeda, urutan data yang berubah, atau environment komputasi yang berbeda.
- **Bagian dari etika penelitian**: riset ML yang tidak reproducible sulit diaudit dan rentan terhadap klaim berlebihan (overclaiming).

Analogi: *resep masakan yang hasilnya hanya enak jika dimasak oleh satu orang tertentu, pada dapur tertentu, hari tertentu, bukan resep yang baik, karena tidak bisa diulang oleh orang lain.*

Prinsip: jika eksperimen tidak bisa dijalankan ulang dengan hasil yang sama, hasil itu belum bisa diklaim sebagai temuan.

g Tiga Pilar Reproducibility

Random Seed

Set random_state / seed di SETIAP langkah berbasis acak: split data, inisialisasi model, shuffle, augmentasi.

```
train_test_split(X, y, random_state=42)
RandomForestClassifier(random_state=42)
```

Environment

Catat versi persis semua library yang dipakai agar environment dapat dibangun ulang identik.

```
pip freeze > requirements.txt
# atau: environment.yml (conda)
```

Data Versioning

Simpan versi/snapshot dataset yang dipakai, terkadang data yang berubah seiring waktu bisa membuat hasil sulit diulang.

```
# contoh: hash file data
md5sum train.csv >> data_manifest.txt
```

Note: beberapa operasi GPU (mis. cuDNN) bersifat non-deterministik secara default, menyebabkan reproducibility 100% kadang perlu trade-off kecepatan vs konsistensi.

g Mencatat Setiap Eksperimen Secara Terstruktur

EXPERIMENT TRACKING

- Setiap kali menjalankan eksperimen (ubah hyperparameter, fitur, atau data), catat konfigurasinya DAN hasilnya, jangan hanya mengandalkan ingatan.
- Minimal yang perlu dicatat: tanggal, versi data, hyperparameter, metrik hasil (train & validation), dan catatan singkat (insight / anomali).
- Untuk proyek lebih besar, pertimbangkan tool experiment tracking (mis. MLflow, Weights & Biases).

Run	Model	Fitur	Seed	CV Score	Catatan
#014	RF (300 trees)	30 fitur (semua)	42	0.958 ± 0.012	Baseline model
#015	RF (300 trees)	24 fitur (RFE)	42	0.971 ± 0.009	Feature selection naikkan skor
#016	LogReg	24 fitur (RFE)	42	0.986 ± 0.006	Terbaik sejauh ini

Manfaat langsung: log seperti ini membuat kita bisa membandingkan eksperimen secara objektif, bukan mengandalkan memori yang bisa keliru.

g Model Card: Mendokumentasikan Model

- **Tujuan & konteks**: masalah apa yang diselesaikan model, untuk siapa, dan batasan penggunaannya.
- **Data**: sumber data, ukuran, periode pengambilan, dan potensi bias yang diketahui.
- **Metodologi**: algoritma, fitur yang dipakai, strategi validasi, baseline pembanding.
- **Hasil & evaluasi**: metrik utama (dengan interval ketidakpastian), hasil error analysis, performa per subgrup.
- **Keterbatasan & risiko**: kondisi di mana model diperkirakan berkinerja buruk; rekomendasi pemantauan setelah deployment.

Standar industri: model card (Google, Mitchell et al. 2019) kini menjadi praktik umum sebelum model ML dipakai di produksi bukan dokumen opsional.

g Checklist Reproducibility Sebelum Menyerahkan Proyek

- Semua random_state di-set secara eksplisit dan konsisten di seluruh notebook/script.
- requirements.txt atau environment.yml disertakan dan sesuai dengan environment yang benar-benar dipakai.
- Kode dapat dijalankan dari awal hingga akhir tanpa error pada mesin yang bersih (uji “restart & run all”).
- Hasil akhir (metrik, visualisasi) konsisten antar run pada seed yang sama.
- Model card / dokumentasi ringkas disertakan: data, metodologi, hasil, keterbatasan.
- Log eksperimen (minimal beberapa run penting) disertakan untuk menunjukkan proses iterasi, bukan hanya hasil akhir.

Reproducibility bukan langkah tambahan di akhir proyek, biasakan untuk lakukan **Reproducibility** sejak baris kode pertama ditulis.

h RINGKASAN

Ringkasan Pertemuan 7

a **Filter, Wrapper, Embedded: tiga strategi memilih fitur relevan**

b **Model-Based Selection: Gini vs permutation importance**

c **Baseline Model: pembanding wajib sebelum klaim “model bagus”**

d **Bias-Variance: learning curve & validation curve sebagai alat diagnosis**

e **Data Leakage: target, train-test, group, dan temporal leakage**

f **Error Analysis: mengubah kesalahan jadi arah perbaikan konkret**

g **Reproducibility: seed, environment, data versioning, model card**

“model yang baik adalah model yang fiturnya terpilih dengan alasan, skornya dibandingkan dengan baseline, kompleksitasnya terdiagnosis lewat bias-variance, skornya bebas dari leakage, kesalahannya dianalisis, dan seluruh prosesnya dapat diulang orang lain”

MELANGKAH KE DEPAN

Dari Pertemuan 7 ke Pertemuan Berikutnya

Benang merah

Pertemuan 7 melengkapi “toolkit diagnosis” : Cara memilih fitur, membandingkan model secara adil, mendiagnosis bias-variance, mendeteksi leakage, menganalisis error, dan mendokumentasikan eksperimen secara reproducible. Toolkit ini akan dipakai berulang kali sepanjang proyek akhir mata kuliah.

- Terapkan checklist Bagian E (leakage) dan Bagian G (reproducibility) pada setiap eksperimen proyek akhir mulai sekarang — bukan hanya di akhir proyek.
- Gunakan learning/validation curve (Bagian D) setiap kali model terlihat “terlalu bagus” atau “kurang bagus” sebelum menyimpulkan apa pun.
- Error analysis (Bagian F) sebaiknya dilakukan setiap kali model baru dilatih.

Diagnosis model yang baik mengubah machine learning dari “trial-and-error” menjadi proses yang disiplin dan dapat dipertanggungjawabkan.

REFERENSI

Bacaan Lanjutan

Topik	Sumber
Buku utama	Aurélien Géron, Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow, 3rd Ed., O'Reilly Media, 2022.
Buku pendamping (statistik & teori)	Gareth James, Daniela Witten, Trevor Hastie, Robert Tibshirani, An Introduction to Statistical Learning with Applications in Python (ISLR), Springer, 2023.
Feature selection	scikit-learn User Guide — Feature selection (sklearn.feature_selection); Géron, Bab 2 & Apx. B.
Permutation importance	scikit-learn User Guide — Permutation feature importance (sklearn.inspection).
Bias-variance & learning curve	ISLR, Bab 2 (Assessing Model Accuracy) & Bab 5 (Resampling Methods); Géron, Bab 4.
Data leakage	scikit-learn User Guide — Common pitfalls and recommended practices; Géron, Bab 2.
Error analysis	Andrew Ng, Machine Learning Yearning, Bab 16–20 (deeplearning.ai).
Reproducibility & model documentation	Mitchell et al., "Model Cards for Model Reporting", FAT* 2019; scikit-learn — Glossary: random_state.
Dataset	Breast Cancer Wisconsin (Diagnostic) — UCI ML Repository / sklearn.datasets; California Housing — dataset dari Géron (2022).